# Partitioning Valid Equivalence Classes

- If elements in a valid equivalence class are not handled in an identical manner by the program, partition the equivalence class into smaller equivalence classes.
- Generate a test case for each valid and invalid equivalence class.

Quadratic equation will be of type:

$$ax^2+bx+c=0$$

Roots are real if $(b^2-4ac)>0$

Roots are imaginary if $(b^2-4ac)<0$

Roots are equal if $(b^2-4ac)=0$

Equation is not quadratic if $a=0$

- Inputs will be integers greater than or equal to 0 and less than or equal to 100

Output domain equivalence class test cases can be identified as follows:

$O_1$={<a,b,c>:Not a quadratic equation if a = 0}

$O_1$={<a,b,c>:Real roots if $(b^2-4ac)>0$}

$O_1$={<a,b,c>:Imaginary roots if $(b^2-4ac)<0$}

$O_1$={<a,b,c>:Equal roots if $(b^2-4ac)=0$}`

| *Test case* | *a* | *b* | *c* | *Expected output* |
|---|---|---|---|---|
| 1 | 0 | 50 | 50 | Not a quadratic equation |
| 2 | 1 | 50 | 50 | Real roots |
| 3 | 50 | 50 | 50 | Imaginary roots |
| 4 | 50 | 100 | 50 | Equal roots |

We may have another set of test cases based on input domain.

$I_1$= {a: a = 0}

$I_2$= {a: a < 0}

$I_3$= {a: 1 ≤ a ≤ 100}

$I_4$= {a: a > 100}

$I_5$= {b: 0 ≤ b ≤ 100}

$I_6$= {b: b < 0}

$I_7$= {b: b > 100}

$I_8$= {c: 0 ≤ c ≤ 100}

$I_9$= {c: c < 0}

$I_{10}$={c: c > 100}

| *Test Case* | *a* | *b* | *c* | *Expected output* |
|---|---|---|---|---|
| 1 | 0 | 50 | 50 | Not a quadratic equation |
| 2 | -1 | 50 | 50 | Invalid input |
| 3 | 50 | 50 | 50 | Imaginary Roots |
| 4 | 101 | 50 | 50 | invalid input |
| 5 | 50 | 50 | 50 | Imaginary Roots |
| 6 | 50 | -1 | 50 | invalid input |
| 7 | 50 | 101 | 50 | invalid input |
| 8 | 50 | 50 | 50 | Imaginary Roots |
| 9 | 50 | 50 | -1 | invalid input |
| 10 | 50 | 50 | 101 | invalid input |

# Let the input values are partitioned as **10 to 20**. and **40 to 60.**

Now we make different can be classified as:

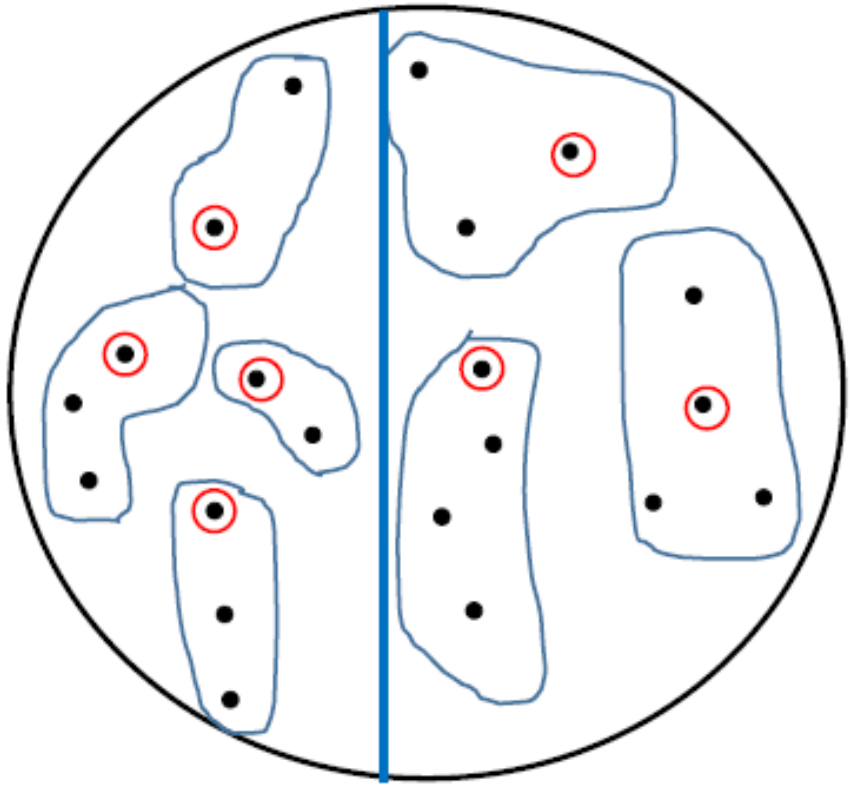
**-- to 9** Invalid.

**10 to 20** Valid.

**21 to 39** Invalid.

**40 to 60** Valid

**61 to ---** Invalid



Now we may select values from each class:

**-7, 16, 31, 49, 88** and see directly whether is it is valid or invalid.

# Example

An integer field shall contain values between and including 1 to 15. By applying equivalence partition what is the minimum number of test cases required for maximum coverage?

- 1
- 2
- 3
- 4

# Example

- In a system designed to work out the tax to be paid
  - An employee has 4000 Rs of salary tax free .The next 1500 is taxed at 10%. The next 28000 Rs is taxed at 22%. Any further amount is taxed at 40%. Which of these groups will fall into same equivalence class?

1. 4800;14000;28000
2. 5200;5500;28000
3. 28001;32000;35000
4. 5800;28000;32000

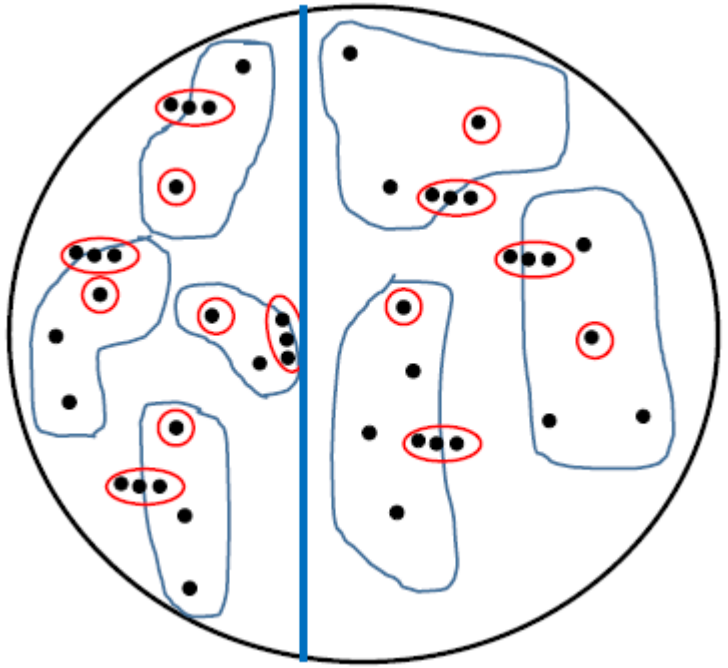
| Valid<br>1-4000 | Valid<br>4001-5500 | Valid<br>5501-33500 | Valid<br>33501≤ |
|---|---|---|---|
| | | | |

# *Boundary Value Analysis*

- When choosing values from an equivalence class to test, use the values that are most likely to cause the program to fail.
- For reasons that are not completely clear, a greater number of errors tends to occur at the boundaries of the input domain rather than in the "center.“
- It is for this reason that *boundary value analysis* (BVA) has been developed as a testing technique. Boundary value analysis leads to a selection of test cases that exercise bounding values.

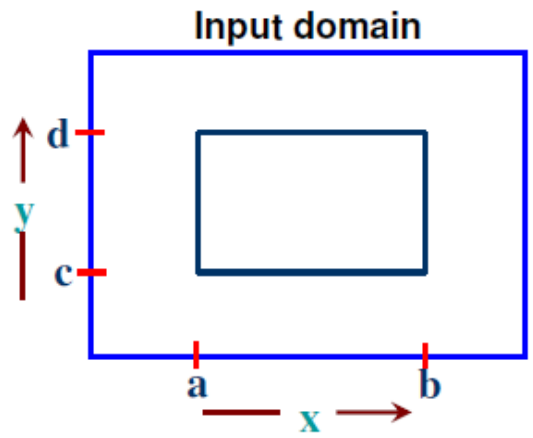
- Boundary value analysis is a test case design technique that complements equivalence partitioning.
- Rather than selecting any element of an equivalence class, BVA leads to the selection of test cases at the "edges" of the class. Rather than focusing solely on input conditions, BVA derives test cases from the output domain as well.
- In addition to testing center values, we should also test boundary values

1. **Right on a boundary**
2. **Very close to a boundary on either side**

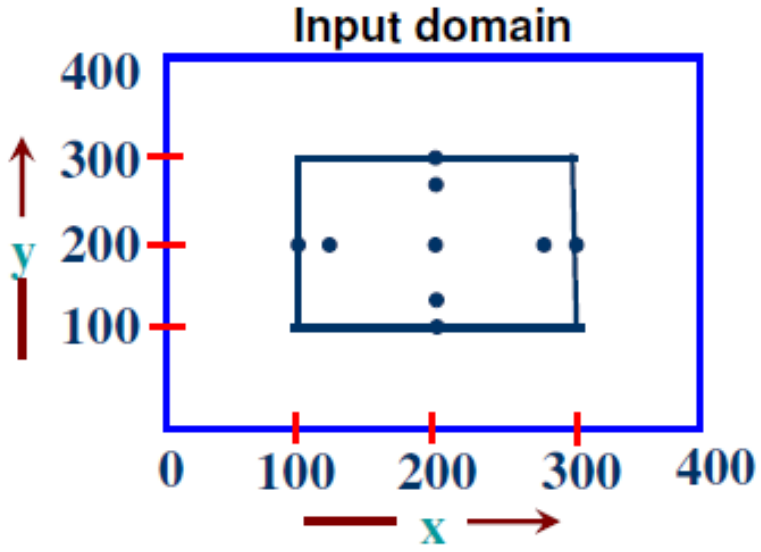


Consider a program with two input variables x and y. These input variables have specified boundaries as:

$$a \leq x \leq b$$
$$c \leq y \leq d$$




- The boundary value analysis test cases for our program with two inputs variables (x and y) that may have any value from 100 to 300 are: (200,100), (200,101), (200,200), (200,299), (200,300), (100,200), (101,200), (299,200) and (300,200).

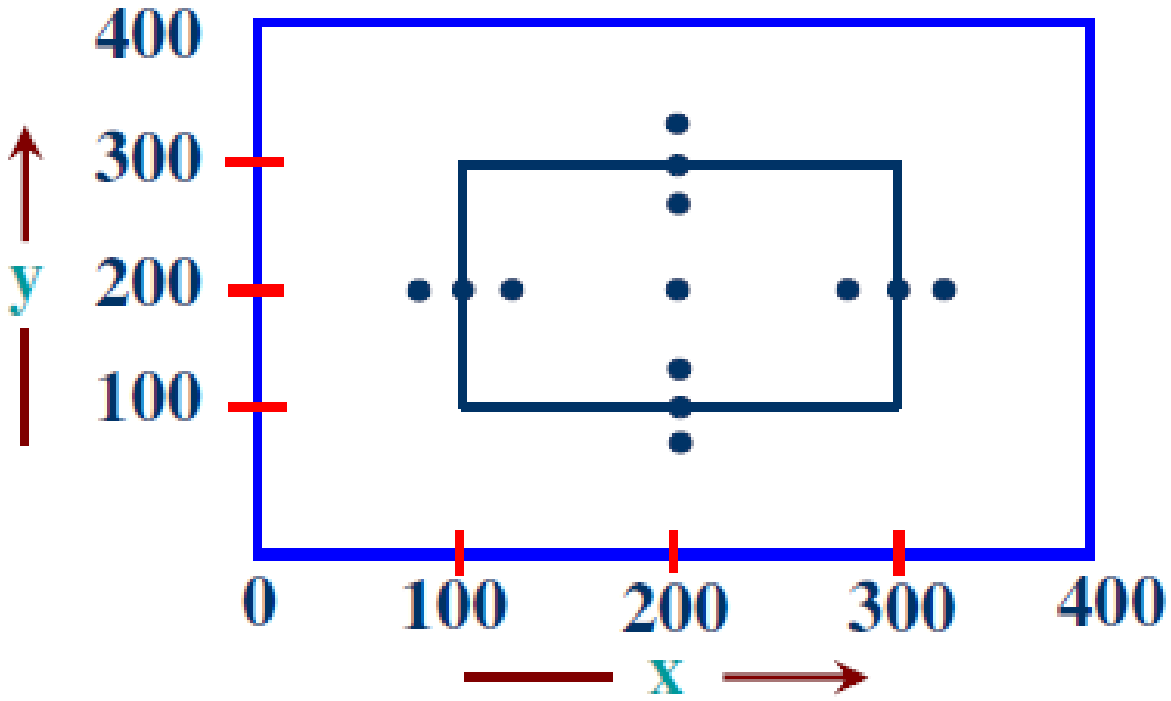



For a program of n variables, boundary value analysis yield **4n + 1** test cases.

# Robustness testing

- It is nothing but the extension of boundary value analysis.
- Here, we would like to see, what happens when the extreme values are exceeded with a value slightly greater than the maximum, and a value slightly less than minimum.
- It means, we want to go outside the legitimate boundary of input domain. This extended form of boundary value analysis is called robustness testing.
- There are additional test cases which are outside the legitimate input domain. Hence total test cases in robustness testing are 6n+1, where n is the number of input variables.

# Robustness testing



Consider the square root problem . Generate robust and test cases for this problem

| Test case | a | b | c | Expected Output |
|---|---|---|---|---|
| 1 | -1 | 50 | 50 | Invalid input` |
| 2 | 0 | 50 | 50 | Not quadratic equation |
| 3 | 1 | 50 | 50 | Real roots |
| 4 | 50 | 50 | 50 | Imaginary roots |
| 5 | 99 | 50 | 50 | Imaginary roots |
| 6 | 100 | 50 | 50 | Imaginary roots |
| 7 | 101 | 50 | 50 | Invalid input |
| 8 | 50 | -1 | 50 | Invalid input |
| 9 | 50 | 0 | 50 | Imaginary roots |
| 10 | 50 | 1 | 50 | Imaginary roots |
| 11 | 50 | 99 | 50 | Imaginary roots |
| 12 | 50 | 100 | 50 | Equal roots |
| 13 | 50 | 101 | 50 | Invalid input |
| 14 | 50 | 50 | -1 | Invalid input |
| 15 | 50 | 50 | 0 | Real roots |
| 16 | 50 | 50 | 1 | Real roots |
| 17 | 50 | 50 | 99 | Imaginary roots |
| 18 | 50 | 50 | 100 | Imaginary roots |
| 19 | 50 | 50 | 101 | Invalid input |

Consider a simple program to classify a triangle. Its inputs is a triple of positive integers (say x, y, z) and the date type for input parameters ensures that these will be integers in the range [1,100]. The program output may be one of the following words:

Expected Output: [Scalene; Isosceles; Equilateral; Not a triangle]

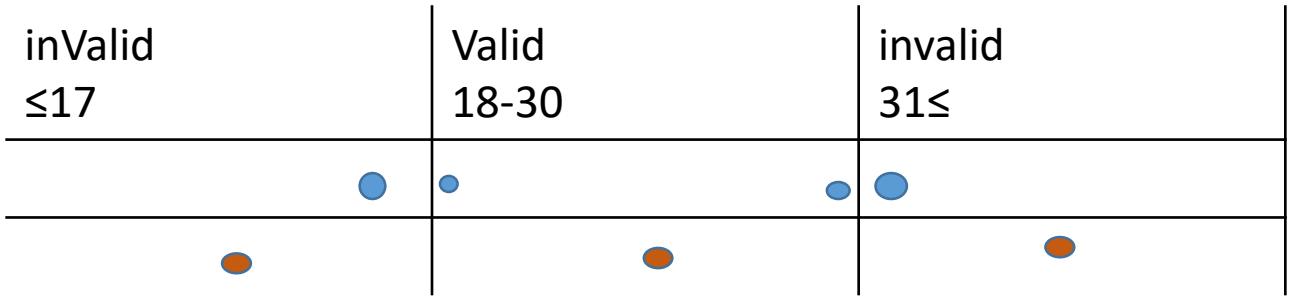
| *Test case* | *x* | *y* | *z* | *Expected Output* |
|---|---|---|---|---|
| 1 | 50 | 50 | 1 | Isosceles |
| 2 | 50 | 50 | 2 | Isosceles |
| 3 | 50 | 50 | 50 | Equilateral |
| 4 | 50 | 50 | 99 | Isosceles |
| 5 | 50 | 50 | 100 | Not a triangle |
| 6 | 50 | 1 | 50 | Isosceles |
| 7 | 50 | 2 | 50 | Isosceles |
| 8 | 50 | 99 | 50 | Isosceles |
| 9 | 50 | 100 | 50 | Not a triangle |
| 10 | 1 | 50 | 50 | Isosceles |
| 11 | 2 | 50 | 50 | Isosceles |
| 12 | 99 | 50 | 50 | Isosceles |
| 13 | 100 | 50 | 50 | Not a triangle |

# Robustness Testing

| ` | x | y | z | Expected Output |
|---|---|---|---|---|
| 1 | 50 | 50 | 0 | Invalid input` |
| 2 | 50 | 50 | 1 | Isosceles |
| 3 | 50 | 50 | 2 | Isosceles |
| 4 | 50 | 50 | 50 | Equilateral |
| 5 | 50 | 50 | 99 | Isosceles |
| 6 | 50 | 50 | 100 | Not a triangle |
| 7 | 50 | 50 | 101 | Invalid input |
| 8 | 50 | 0 | 50 | Invalid input |
| 9 | 50 | 1 | 50 | Isosceles |
| 10 | 50 | 2 | 50 | Isosceles |
| 11 | 50 | 99 | 50 | Isosceles |
| 12 | 50 | 100 | 50 | Not a triangle |
| 13 | 50 | 101 | 50 | Invalid input |
| 14 | 0 | 50 | 50 | Invalid input |
| 15 | 1 | 50 | 50 | Isosceles |
| 16 | 2 | 50 | 50 | Isosceles |
| 17 | 99 | 50 | 50 | Isosceles |
| 18 | 100 | 50 | 50 | Not a triangle |
| 19 | 101 | 50 | 50 | Invalid input |

- A test field in an application accepts inputs as the age of user. Here, the values are allowed to be accepted by the field is between 18 to 30 years, both values inclusive. By applying EP and BVA , which of the given option consists of valid boundary values and valid equivalence value.

1. 17,18,20
2. 18, 30, 25
3. 18,30,31
4. 19,20,31

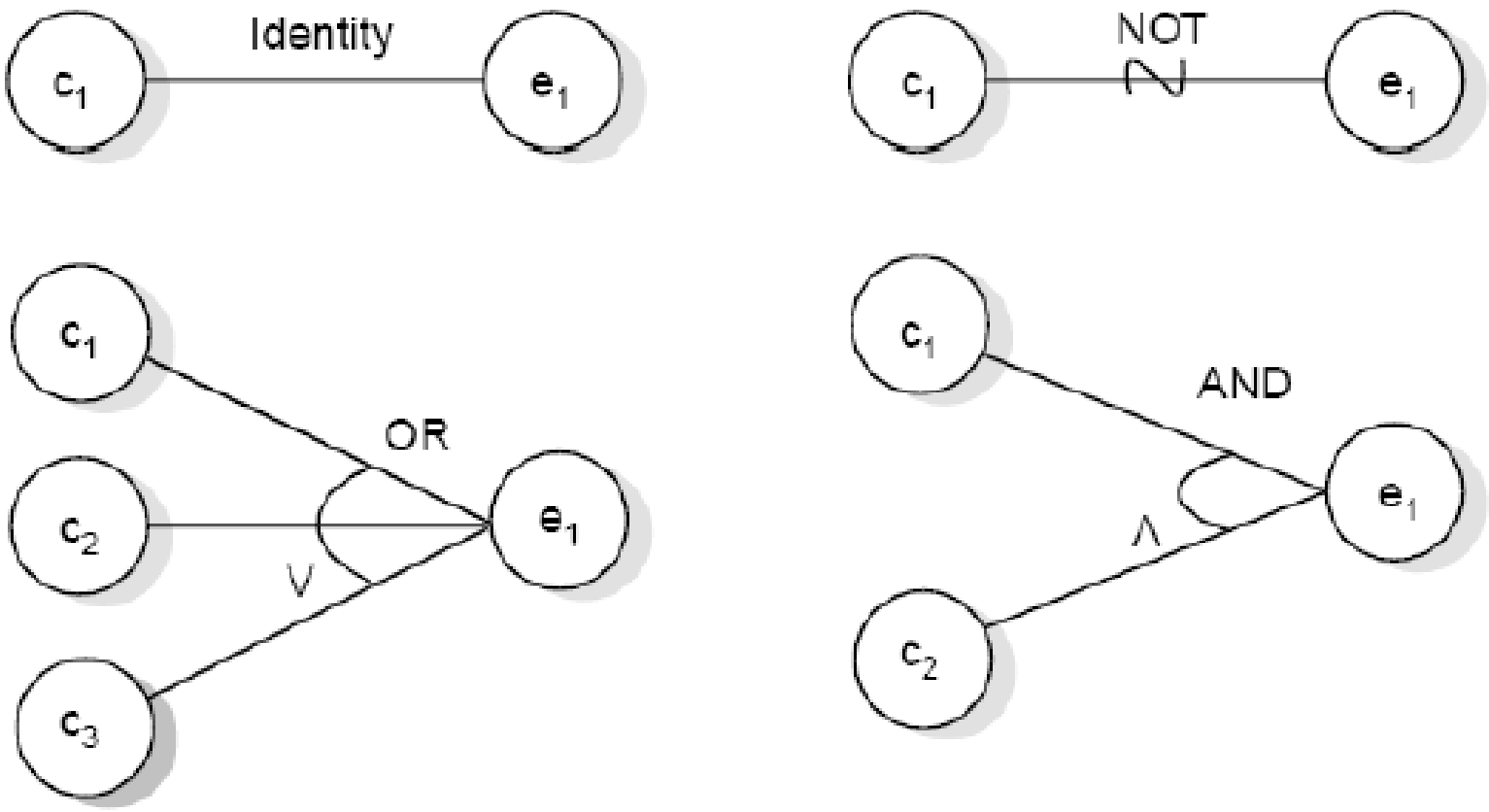


# Cause Effect Graphing

- One weakness with the equivalence class partitioning and boundary value analysis method is that they consider each input separately.
- That is both concentrate on conditions and classes of one input.
- They do not consider combinations of input circumstances that may form interesting situations that should be tested.
- Cause Effect Graphing  is a technique that aids in selecting combinations of input conditions in a systematic way such that the number of test cases does not become unmanageably large.
- The technique starts with identifying causes and effects of the system under testing.

- A cause is a distinct input condition and an effect is a distinct output condition.
- Each condition forms a node in the cause-effect graph.
- The condition should be stated such that they can be set to either true or false.
- After identifying the causes and effects , for each effect we identify the causes that can produce that effect and how the conditions have to be combined to make the effect true.
- Conditions are combined using Boolean operators: “and (^)”, “or (V)” and “not(~)”.
- A test case is generated for each combination of conditions which make some effect true.

# Basic cause effect graph symbols



Myers explained this effectively with following example. "The characters in column 1 must be an A or B. The character in column 2 must be a digit. In this situation, the file update is made. If the character in column 1 is incorrect, message *x* is issued. If the character in column 2 is not a digit, message y is issued".

| S.No | 1 | 2 | 3 |
|---|---|---|---|
| C1 | T | F | X |
| C2 | T | F | X |
| C3 | T | X | F |
| E1 | **T** | | |
| E2 | | **T** | |
| E3 | | | **T** |

**The causes are**

$c_1$: character in column 1 is A

$c_2$: character in column 1 is B

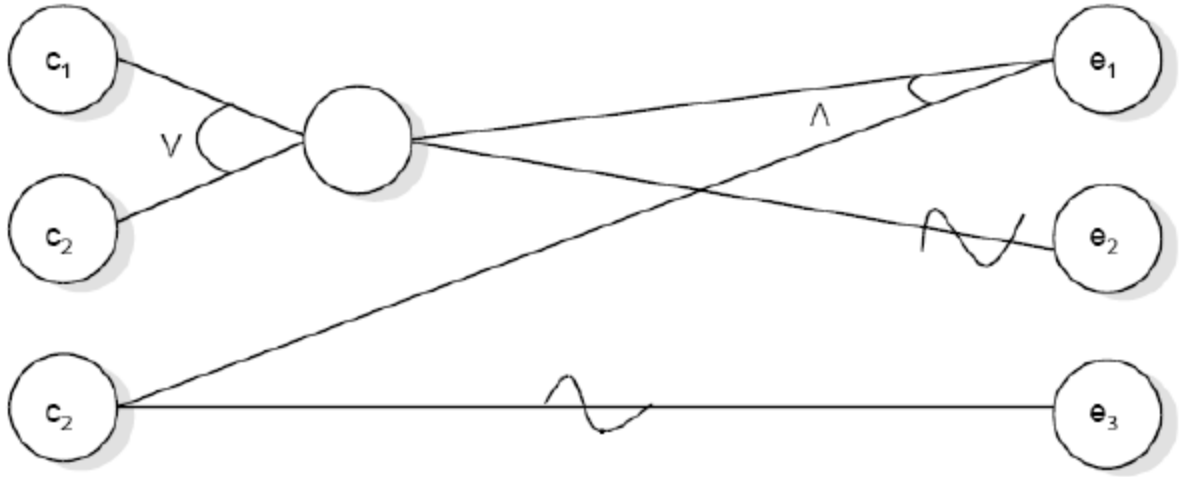
$c_3$: character in column 2 is a digit

**and the effects are**

$e_1$: update made

$e_2$: message *x* is issued

$e_3$: message *y* is issued



# TRIANGLE PROBLEM

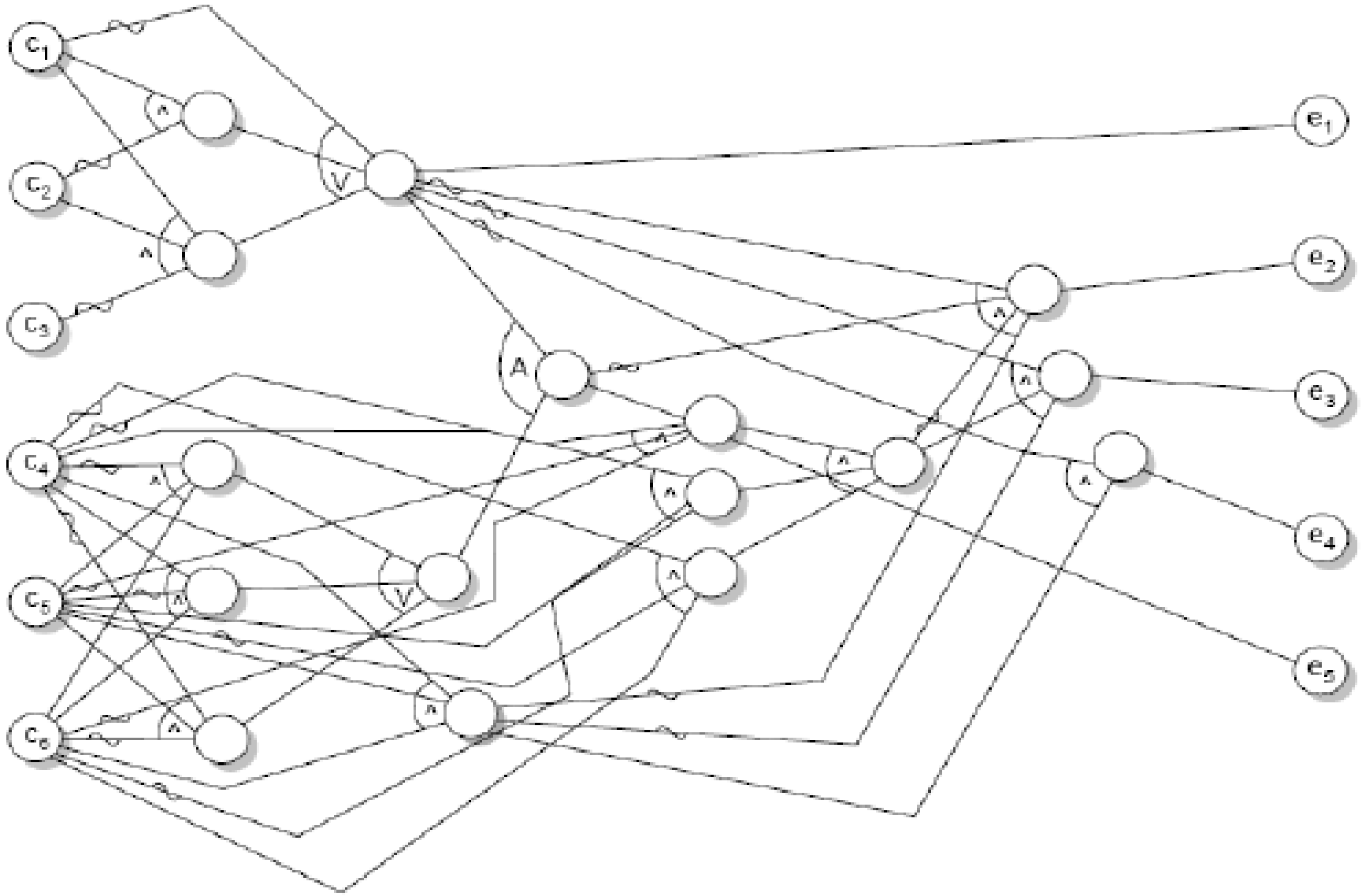
The causes are

- $c_1$: side x is less than sum of sides *y* and *z*
- $c_2$: side y is less than sum of sides *x* and *y*
- $c_3$: side z is less than sum of sides *x* and *y*
- $c_4$: side x is equal to side *y*
- $c_5$: side x is equal to side *z*
- $c_6$: side y is equal to side *z*

and effects are

- $e_1$: Not a triangle
- $e_2$: Scalene triangle
- $e_3$: Isosceles triangle
- $e_4$: Equilateral triangle
- $e_5$: Impossible stage

| Conditions | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $C_1$: x < y + z ? | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| $C_2$: y < x + z ? | X | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| $C_3$: z < x + y ? | X | X | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| $C_4$: x = y ? | X | X | X | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |
| $C_5$: x = z ? | X | X | X | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 |
| $C_6$: y = z ? | X | X | X | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 |
| $e_1$: Not a triangle | 1 | 1 | 1 | | | | | | | | |
| $e_2$: Scalene | | | | | | | | | | | 1 |
| $e_3$: Isosceles | | | | | | | 1 | | 1 | 1 | |
| $e_4$: Equilateral | | | | 1 | | | | | | | |
| $e_5$: Impossible | | | | | 1 | 1 | | 1 | | | |


c1
c2
c3
c4
c5
c6
e1
e2
e3
e4
e5

# SOFTWARE TESTING STRATEGIES

- UNIT TESTING
- INTEGRATION TESTING
- SYSTEM TESTING

# UNIT TESTING

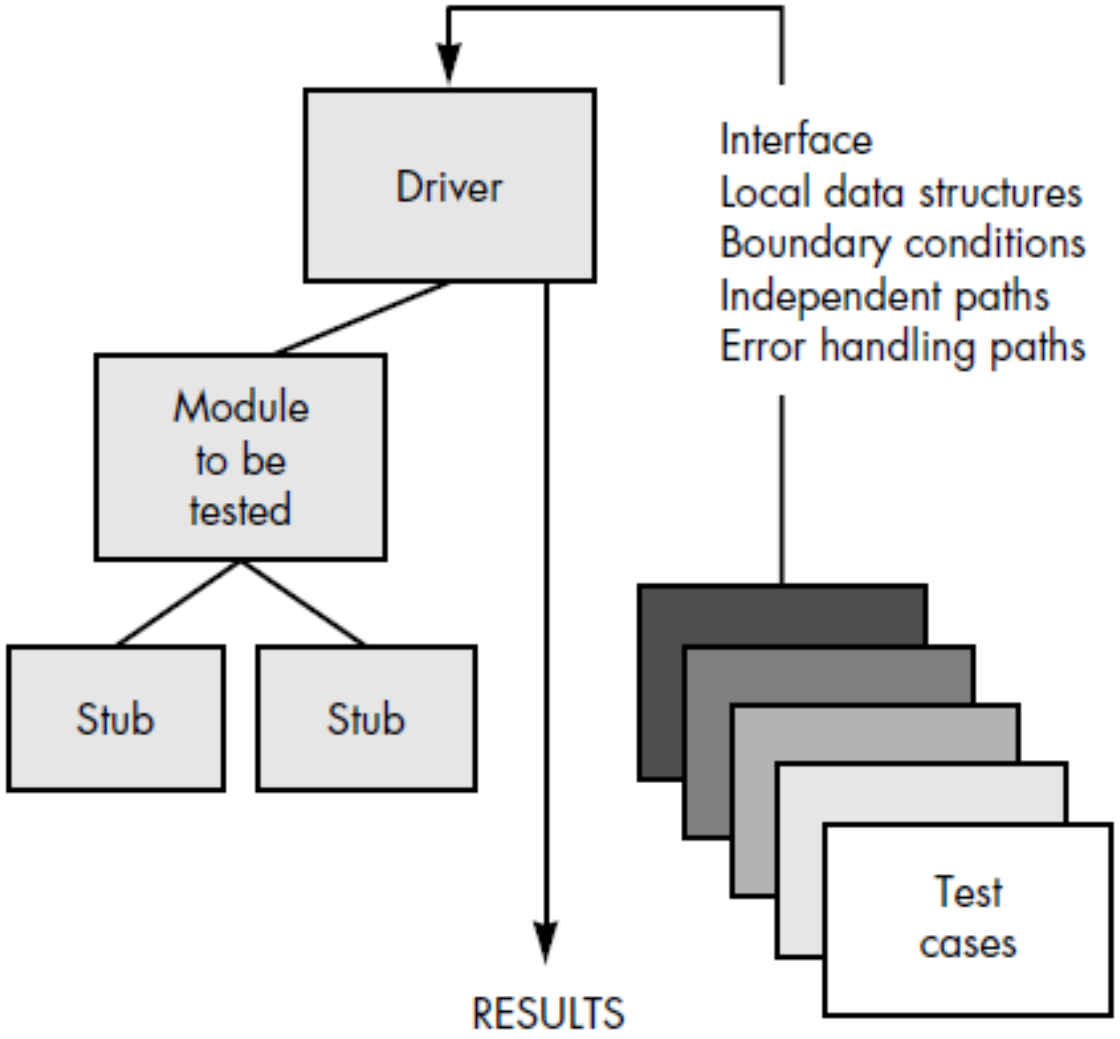
- Unit testing focuses verification effort on the smallest unit of software design—the software component or module.
- Using the component-level design description as a guide, important control paths are tested to uncover errors within the boundary of the module.
- The relative complexity of tests and uncovered errors is limited by the constrained scope established for unit testing. The unit test is white-box oriented, and the step can be conducted in parallel for multiple components

# Unit Test Considerations

- The module interface is tested to ensure that information properly flows into and out of the program unit under test.
- The local data structure is examined to ensure that data stored temporarily maintains its integrity during all steps in an algorithm's execution.
- Boundary conditions are tested to ensure that the module operates properly at boundaries established to limit or restrict processing.
- All independent paths (basis paths) through the control structure are exercised to ensure that all statements in a module have been executed at least once.
- And finally, all error handling paths are tested.

In Unit Testing, Test cases should uncover errors such as

1. Comparison of different data types
2. Incorrect logical operators or precedence,
3. Expectation of equality when precision error makes equality unlikely,
4. Incorrect comparison of variables,
5. Improper or nonexistent loop termination,
6. Failure to exit when divergent iteration is encountered,
7. improperly modified loop variables.




In most applications a *driver* is nothing more than a "main program" that accepts test case data, passes such data to the component (to be tested), and prints relevant results.

*Stubs* serve to replace modules that are subordinate (called) by the component to be tested. A stub or "dummy subprogram" uses the subordinate module's interface, may do minimal data manipulation, prints verification of entry, and returns control to the module undergoing testing.